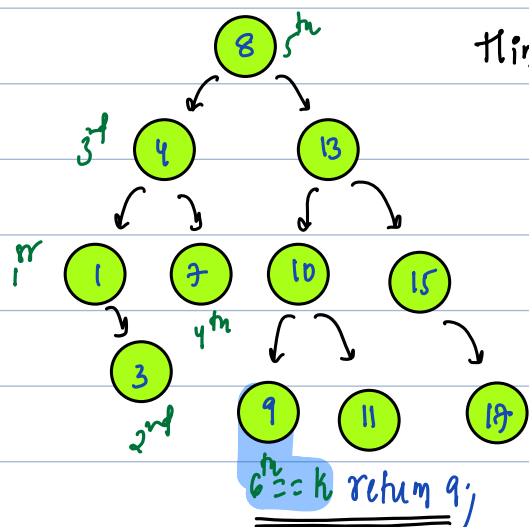


Today's Content

1. k^{th} Smallest in BST ✓
2. Morris InOrder traversal
3. LCA: Least Common Ancestor
4. LCA in BST

Q: Finding the k^{th} smallest element in a BST.

$k = 6^{\text{th}}$ smallest.



Hint: In BST inorder is in increasing order.

Idea1: Store inorder in an array & return $arr[k-1]$;

TC: $O(N)$ SC: $O(N)$? {Store entire array}

Idea2: While printing inorder keep a counter variable
if counter == k: That element is k^{th} smallest element.

int k^{th} smallest(Node root, int k) { TC: $O(N)$ SC: $O(H)$

Node curr = root;

int c = 1;

Stack<Node> st = new Stack<>();

while ((st.size() > 0) || (curr != null)) {

while (curr != null) {

st.push(curr);

curr = curr.left;

curr = st.pop();

st.pop();

if (c == k) { break; }

c++;

curr = curr.right;

}

return curr.data

Morris Inorder Traversal:

TODO: Morris PreOrder & PostOrder:

Concept can be applied to any B.T. { Binary Tree }

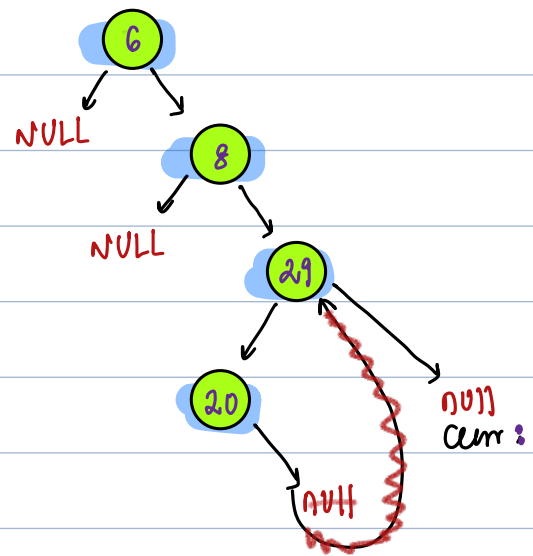
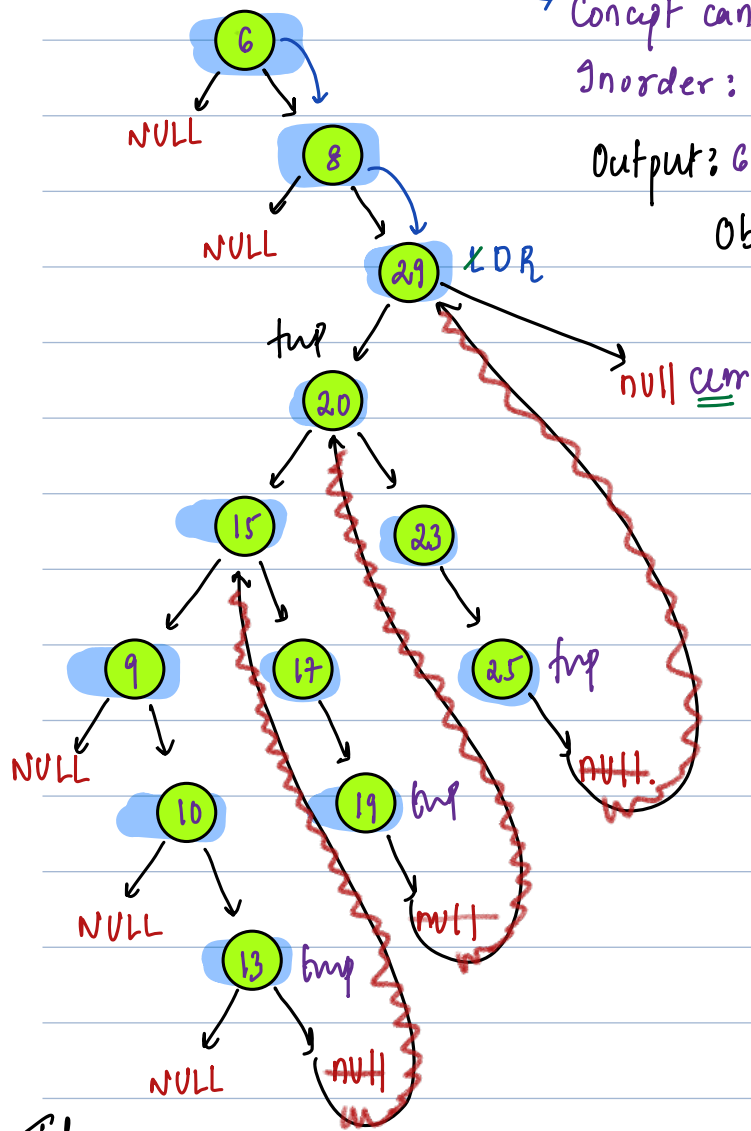
Inorder: LDR

Output: 6 8 9 10 13 15 17 19 20 23 25 29

Obs: In Inorder of left node is Present

1. If node visited 1st time = goto left

2. If node visited 2nd time = print & goto right



Idea:

Repeat till $curr != null$;

1. If $curr.left == null$;

print $curr.data$ & goto right

2. $curr.left != null$;

Node $temp = curr.left$; Get max of left.

Keep updating $temp$ till reaches max;

if ($temp.right == null$) { // 1st time; LDR // max $right == null$ means: 1st

$temp.right = curr$; $curr = curr.left$; // Attach link & $curr$ to left

else { // $temp.right == curr$ 2nd time LDR // max $right == curr$: 2nd time

$temp.right = null$; // Detach link & print $curr$ node & goto right

print ($curr.data$); $curr = curr.right$

void inorder(Node root) { Tc: $O(N)$ sc: $O(1)$ // 8:25 AM

Node curr = root;

while (curr != null) {

if (curr.left == null) { // ~~X~~ DR

print(curr.data);

curr = curr.right;

else { // have left, calculate man on left side

& if man right == null: 1st, man right == curr: 2nd;

Node tmp = curr.left;

while (tmp.right != null && tmp.right != curr) {

tmp = tmp.right;

if (tmp.right == null) { // 1st time; ~~X~~ DR

tmp.right = curr;

curr = curr.left;

else { // tmp.right == curr 2nd time ~~X~~ DR

tmp.right = null; // detach link

print(curr.data);

curr = curr.right;

}

}

}

}

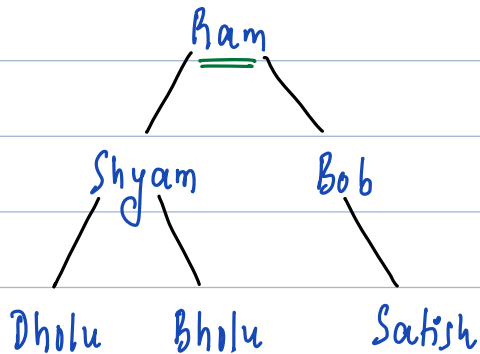
We are all connected.

It is said that we all humans are related through some common ancestor at some point.

Given the binary tree T representing the family tree.

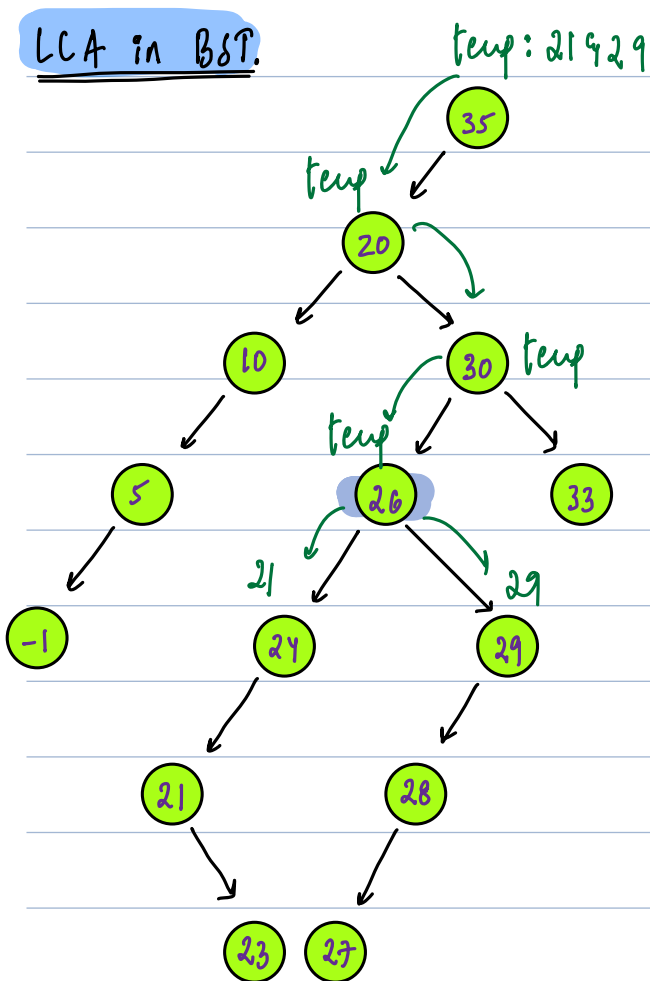
Discover the earliest family member connects two given people B and C in a family tree.

Ex:



LCA between Dholu & Bob = ? Ram:

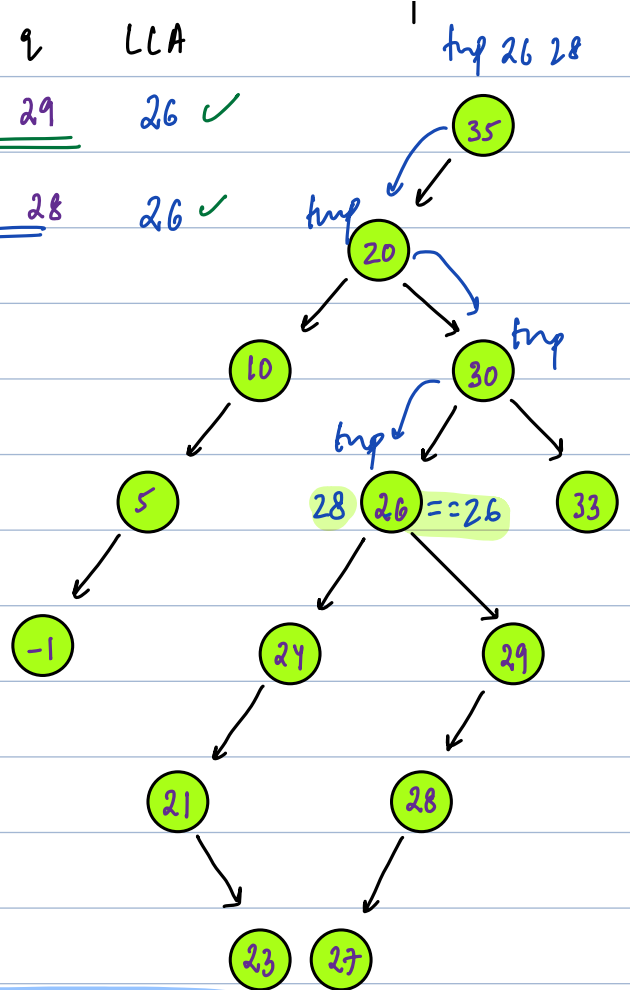
LCA in BST.



P 9 LCA

En1: 21 29 26 ✓

En3: 26 28 26 ✓



Note: In Question it is mentioned that both p & q are present.

Idea:

↳ you need to ask follow up questions to interviewer.

grace marks

1. Temp = root;

At each step;

if Temp < both values;

Temp goes to right

else if Temp > both values;

Temp goes to left

else

Case: Both values are on either side of Temp; branching

Case: If one value = temp;

Ancestry starts at temp;

break

}

return temp;

int LCA(Node root, int B, int C) { Tc: $O(H)$ sc: $O(1)$

Node tmp = root;

while (tmp != null) {

if (tmp.data < B && tmp.data < C) {

tmp = tmp.right;

else if (tmp.data > B && tmp.data > C) {

tmp = tmp.left;

else {

// Case 1: B & C are going to opposite sides of tmp

// Case 2: One of B, C are equal to tmp;

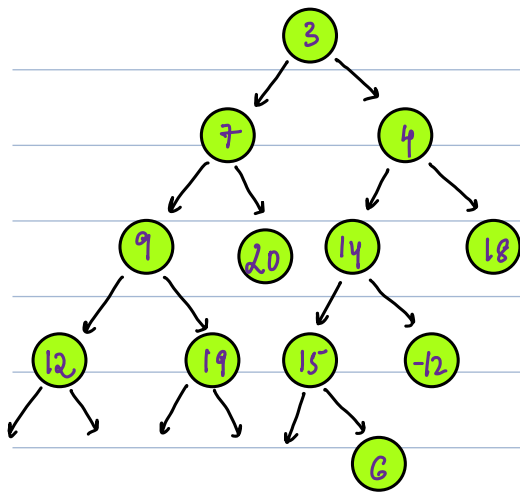
Note: In both above cases ancestor is tmp;

break;

}

return tmp;

}



	B	C	LCA
En1:	15	18	4
En2:	-12	4	4
En3:	9	19	9
En4:	20	19	7
En5:	15	-12	14

Idea: Recursively search both values B & C.

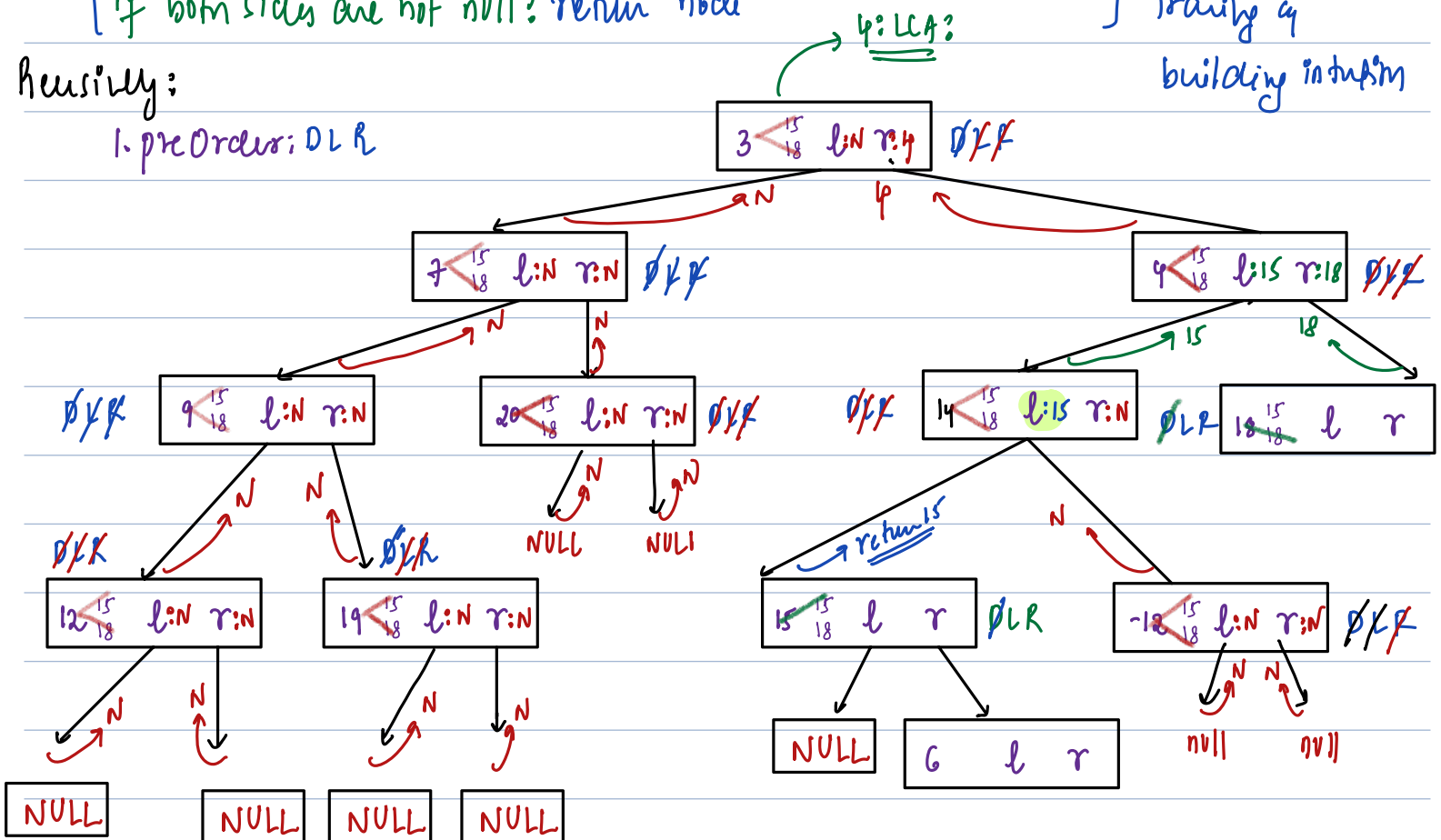
if node matches with one of values B, C: **return node**

if nothing matching: **return null**.

[if one side is null & other side is not: return not null side.] Comes from
[if both sides are not null: return node] Tracing & building intuition

Recursively:

1. preOrder: DLR



Note: While search for a, adding those nodes, which return true.
All those nodes will form path.

Idea: Recursively search both values B & C.

if node matches with one of values B, C: **return node**

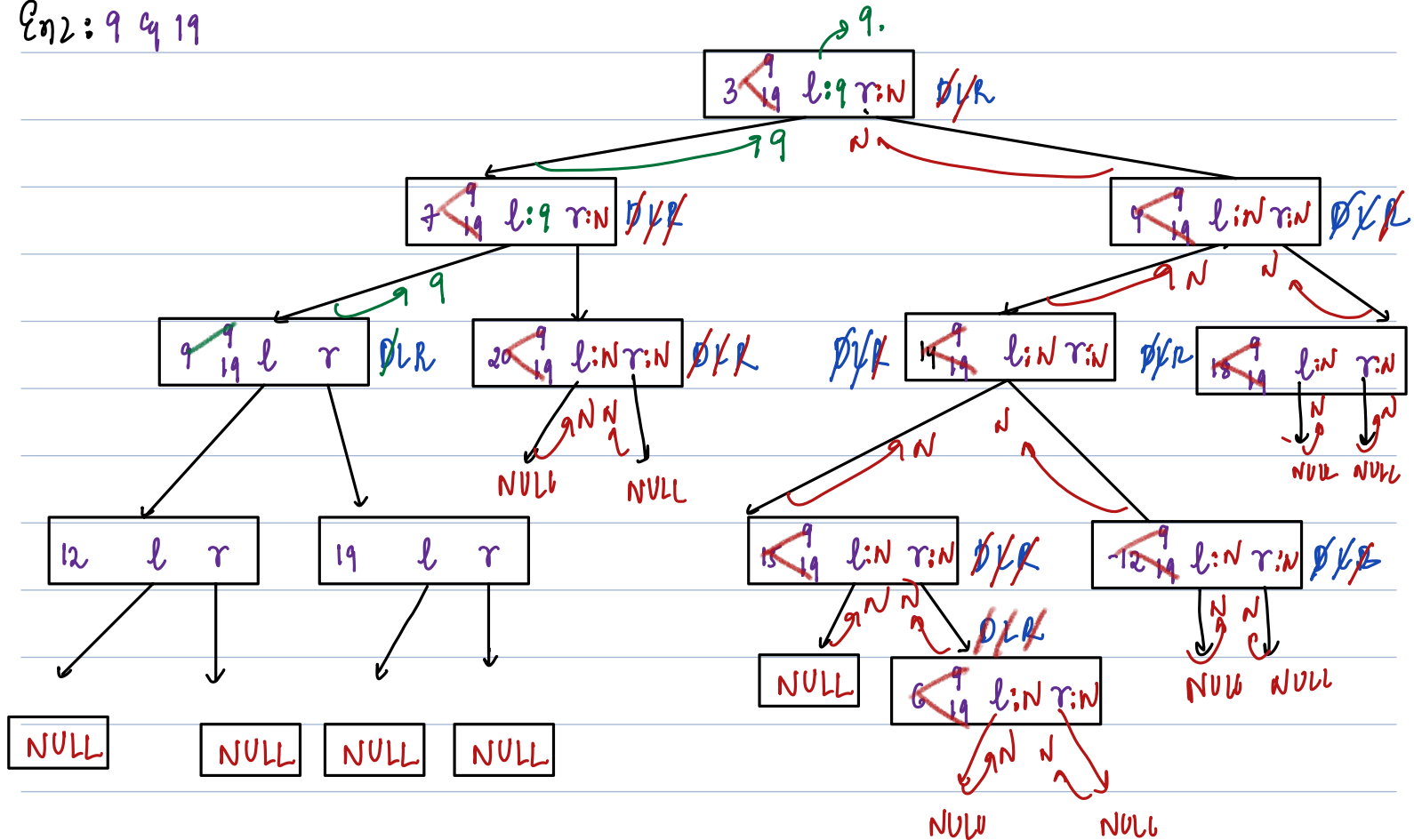
if nothing matching, reached null : **return null.**

if one side is null & other side is not: return not null side.

if both sides are not null: return node

TC: $O(N)$ SC: $O(H)$

Ex2: 9 & 19



Node BTLLCA(Node root, int B, int C) { TC: $O(N)$ SC: $O(H)$

```
if (root == null) { return null; }
```

```
if (root.data == B || root.data == C) { D L R
    return root;
}
```

```
Node leftLCA = BTLLCA(root.left, B, C);
```

```
Node rightLCA = BTLLCA(root.right, B, C);
```

```
if (leftLCA != NULL && rightLCA != NULL) {
    return root;
}
```

```
else if (leftLCA != NULL) {
    return leftLCA;
}
```

```
else if (rightLCA != NULL) {
    return rightLCA;
}
```

```
else {
    return null;
}
```

}

Node solve(Node root, int B, int C) {

```
Node LCA = BTLLCA(root, B, C);
```

```
return LCA;
}
```

}